

26-Arch Lab+ 实验报告

潘孝圆 24300240128

2026-06-18

1 完成内容

本次 Lab+ 在已有 Lab1-Lab6 实现基础上继续补充 bonus。主要完成内容如下：

- 保留并验证 Lab1 extra 的乘除法扩展支持。
- 保留 Nexys4 上板适配，继续使用非 Basys3 开发板配置。
- 保留 Lab5 的 Sv39 MMU、2 MiB/1 GiB hugepage 支持、特权级和 Lab6 异常中断实现。
- 新增 Lab+ atomic extension：实现 AMO W 系列指令以及 LR.W/SC.W，并接入 difftest atomic event。
- 新增前端性能优化：顺序取指提前发起请求，并为 CBusArbiter 增加 idle fast path，减少固定仲裁空泡。
- 新增 PMP0/privfull 支持：支持 PMP0 的 OFF/TOR/NA4/NAPOT 匹配，产生 instruction/load/store access fault，并通过 lab+/4 privileged sys-test。
- 新增 EBREAK 断点异常支持：SYSTEM/funct12=0x001 触发同步异常 cause 3。
- 新增 FENCE/FENCE.I 合法 no-op 支持，提升编译器生成程序的兼容性。
- 新增 xv6 主线部分进展：补充真实 S-mode、SRET、异常/中断委托和 S 态 trap CSR 写入路径。
- 新增 MMU page fault 与 PTE 权限检查：识别 Sv39 非 canonical 地址、无效 PTE、叶子页权限不满足、巨页 PPN 未对齐，并产生 instruction/load/store page fault。
- 新增 test-labplus-2/3/4 三个 Makefile 测试入口，并补入官方 Lab+ ready-to-run 测试文件。

lab+/4 全量 TEST=all 已完成 benchmark 和 sys-test，最终输出 Privileged test finished. Exit with code = 0。当前官方 all-test-privfull.bin 中未包含真实 ebreak 指令，breakpoint [X] 来自测试程序自身的占位输出；补充 EBREAK 后该输出仍不会变化，不影响最终 privileged 测试收尾。

2 Atomic Extension 设计

2.1 指令解码

新增 opcode 0101111 的 A 扩展解码，当前支持 funct3=010 的 32 位原子指令：AMOADD.W、AMOSWAP.W、AMOXOR.W、AMOAND.W、AMOR.W、AMOMIN.W、AMOMAX.W、AMOMINU.W、AMOMAXU.W、LR.W、SC.W。

LR.W 要求 $rs2=0$ 。所有 AMO W 指令按照 store/AMO 地址对齐规则检查地址低两位，不对齐时产生 store/AMO address misaligned 异常。

2.2 两阶段访存

当前 CPU 是顺序单发结构，同一时刻只有一条指令处于访存阶段。普通 AMO 指令被实现为两阶段状态机：

```
read old word from memory
compute new word
write new word back to same address
commit rd = sign_extend(old word)
```

第一阶段读响应到达后，指令仍保持 $mem_pending=1$ ，并将同一个请求槽切换为写请求。这样不会让取指或下一条指令插入 AMO 的读写之间，满足单核测试中的原子性要求。

LR.W 只执行读请求，并记录 $\{addr[63:2], 2'b00\}$ 作为 reservation 地址。SC.W 在 reservation 命中时发出写请求并返回 0；未命中时不访问内存，直接返回 1，并清除 reservation。

2.3 Difftest 事件

AMO 不能直接作为普通 store event 上报。修复方式如下：

- 普通 DifftestStoreEvent 屏蔽 dt_is_atomic 。
- AMO 访存提交时额外上报 DifftestAtomicEvent。
- AMO load event 的 $fuType$ 设置为 0x0f，使 difftest 按 atomic 路径处理。
- $atomicData$ 使用未移位的 $rs2$ ， $atomicOut$ 使用旧值， $atomicMask$ 根据地址低三位生成。

3 Privfull 与 PMP 支持

3.1 jalr 不对齐异常

$lab+4$ 的 $instr_misalign$ 使用 $jalr$ 跳转到 $target + 1$ 。原实现使用清除 bit0 后的 $jalr$ 目标判断是否不对齐，因此该项不会触发异常。为匹配课程测试，本次改为正常跳转目标仍使用 $\{raw_target[63:1], 1'b0\}$ ，但 instruction address misaligned 判断和 $mtval$ 使用未清 bit0 的 raw_target 。该修改后 Lab6 的 $instr_misalign$ 回归仍通过。

3.2 PMP0 匹配与权限

实现了一个最小 PMP0，用于覆盖 Lab+ privfull 测试：

- 支持 $pmpcfg0[4:3]$ 的 OFF、TOR、NA4、NAPOT 地址匹配。
- 支持 $pmpcfg0[2:0]$ 的 R/W/X 权限检查。
- $pmpcfg0.A=OFF$ 时不影响原有 Lab5/Lab6。
- PMP 激活后，U 态未命中 PMP 或权限不足会触发 access fault。
- M 态在 PMP entry 未 lock 时绕过权限检查。

测试程序设置的 PMP 范围为 0x80006000 到 0x80007000，即 4096B 用户区。PMP 激活后，用户态访问该范围外的数据或跳转到该范围外取指都会触发异常。

3.3 异常接入

取指侧新增 `if_id_access_fault`。当 PMP 拒绝取指时，CPU 不再真的向内存发起请求，而是向 `decode` 注入一条携带 `fault` 信息的占位指令，随后走统一 `trap` 路径，产生 `cause 1`。

数据侧在发起 `dreq` 之前检查 PMP:load/LR/AMO read 权限不足产生 `cause 5`;store/SC/AMO write 权限不足产生 `cause 7`；地址不对齐优先级仍高于 `access fault`。

另外新增 `fetch_priv` 解决 `trap/mret` 重定向同周期的权限判断问题：`trap` 重定向到 `mtvec` 时按 M 态检查取指，`mret` 重定向时按返回后的权限检查取指。

3.4 EBREAK 断点异常

Lab+ `privfull` 输出中存在 `Test breakpoint [X]`。反汇编和二进制字节检查显示当前官方 `all-test-privfull` 没有 `ebreak` 编码 `0x00100073`，该项不会真正触发断点异常。为补齐异常类型，本次仍在 CPU 中新增 `EBREAK`：

- 在 `SYSTEM` 指令 `funct3=000` 且 `instr[31:20]=12'h001` 时识别为 `EBREAK`。
- `EBREAK` 作为合法指令进入同步异常路径。
- `trap cause` 设置为 3，`mtval` 保持 0。
- 已重跑 `make test-labplus-3`、`Lab+ privfull sys-test` 和 `Lab6 sys-test`，均保持通过。

3.5 FENCE/FENCE.I 兼容

当前 CPU 没有 `cache`，也没有乱序访存或 `speculative` 执行，因此普通 `FENCE` 和 `FENCE.I` 不需要真实刷新硬件状态。本次将 `opcode 00011111` 中 `funct3=000/001` 的指令作为合法 `no-op` 提交：

- `FENCE` 用于兼容内存顺序屏障。
- `FENCE.I` 用于兼容指令流同步屏障。
- 两者不写寄存器、不发访存请求、不触发流水线重定向。
- 已随 `atomic`、`Lab+ privfull` 和 `Lab6` 回归一起验证。

3.6 xv6 主线部分进展：S-mode 与委托

官方 Lab+ 主 Track 是尝试运行 `xv6`。当前仓库没有 `xv6` 镜像或源码，因此本次先补 `xv6 boot` 的 CPU 前置能力，而不是直接实现磁盘 MMIO。新增内容如下：

- 新增 `PRIV_S=2'b01`，`mret` 可以返回到 S 态，`diffest` 的 `privilege mode` 也能看到 S 编码。
- 新增 `SRET` 解码，S/M 态可执行，返回到 `sstatus.spp` 指定的 U/S 态，并按规范更新 `sstatus.sie/spie/spp`。
- `ECALL` 在 S 态触发 `cause 9`，U 态仍为 `cause 8`，M 态仍为 `cause 11`。
- 开放 `medeleg` 常见可委托异常位，开放 `mideleg` 的 `SSIP/STIP/SEIP` 位。
- `trap` 被委托到 S 态时，跳转 `stvec`，写入 `sepc/scause/stval`，并更新 `sstatus.sie/spie/spp`；未委托时保持原 M 态 `trap` 路径。
- 增加 S 级中断 `evaluate` 框架：当 `mip/sip`、`mie/sie`、`mideleg` 同时打开时，U/S 态可进入 S `trap`。

这一部分还不能直接跑完整 `xv6`，因为仍缺少 `virtio`/磁盘 MMIO 或替代块设备模型，以及更完整的 `SUM/MXR` 和 `A/D` 位硬件更新。但它把 `xv6` 所需的 Supervisor `trap` 基础路径补上了，并通过现有 `Lab5`、`Lab6` 与 `Lab+ privfull` 回归确认没有破坏原 U/M 行为。

3.7 MMU page fault 与 PTE 权限检查

原 MMU 在页表项无效时会把最终物理地址置 0 并继续发起访存，这对 Lab5 的正常路径足够，但不适合继续做 xv6/异常测试。本次在 CBus 响应中增加 `page_fault` 标记，并把该标记一路传回取指和数据访存路径：

- CBus 请求增加 `is_instr`，MMU 能区分取指、load 读和 store/AMO 写。
- CBus/IBus/DBus 响应增加 `page_fault`，MMU 发现翻译失败时返回一拍 fault 响应，不再访问物理地址 0。
- 取指 page fault 进入 decode 统一异常路径，产生 cause 12，`mtval/stval` 写入 faulting virtual address。
- load/LR/AMO read page fault 在数据响应阶段产生 cause 13。
- store/SC/AMO write page fault 在数据响应阶段产生 cause 15，并阻止该访存提交到 difftest。
- MMU 检查 Sv39 canonical 地址、V=0、W=1 && R=0、第三级仍非叶子、叶子页 R/W/X/U 权限和 1 GiB/2 MiB 巨页 PPN 对齐。
- 为兼容官方 Lab5 kernel，本次不强制 A/D 位。若后续要严格按规范处理，需要实现硬件置 A/D 或让软件 trap 后设置 A/D。

数据侧 page fault 是响应期异常，不在 decode 阶段就能确定。因此本次额外补了 WB trap 重定向：当 `dresp.page_fault=1` 时清除等待中的访存、写入 trap CSR，并清空 IF/ID，防止后续指令越过 faulting load/store 提交。

4 前端性能优化

4.1 顺序取指提前

优化前取指状态机只在以下条件满足时发起新请求：

```
!if_pending && !if_id_valid && !mem_pending
```

优化后新增 `if_can_request`：

```
!if_pending && !mem_pending
&& (!if_id_valid || (id_consume && !id_redirect))
```

当当前 `if_id` 指令在本周期被消费，并且不是 trap、跳转、taken branch、CSR 或 MRET 时，CPU 同周期发起下一条顺序取指请求。对于重定向类指令，仍然等待目标写入 `pc` 后再取指。

4.2 CBusArbiter fast path

原 CBusArbiter 在 idle 状态下不会直接把选中的请求透传到输出总线，因此每个请求都会固定多等一拍。本次改为 idle 且存在请求时，`oreq` 直接等于当前选中的 `selected_req`；如果下游同周期返回 last，仲裁器不进入 busy；如果请求未完成，才锁存 `index` 并进入 busy。该优化对取指和数据访存都生效。

4.3 周期对比

测试	原始周期	顺序提前	加 fast path	总减少	最终 IPC
atomicity.bin	372	317	249	33.1%	0.221
lab1-extra-test.bin	185976	153201	120425	35.2%	0.272
lab4-test.bin	208529	177990	139050	33.3%	0.236

MicroBench qsort 采样从 7 ms 降到 6 ms。lab+/4 TEST=all 中的 benchmark 结果如下：

CoreMark: 738 ms, 13 Iterations/Sec

Dhrystone: 1046 ms, 16 Marks

STREAM Copy/Scale/Add/Triad: 14.5 / 0.9 / 1.8 / 0.8 MB/s

5 代码修改

- `vsrc/src/core.sv`: 新增 AMO 解码、AMO 结果计算、reservation 记录；扩展访存状态机；新增 difftest atomic event；新增 `if_can_request`；新增 PMP0 匹配、取指 access fault 注入、load/store access fault；新增 `fetch_priv`；新增 EBREAK 解码和 breakpoint exception cause 3；新增 FENCE/FENCE.I 合法 no-op 解码；新增 S-mode、SRET、medeleg/mideleg 委托、S 态 trap CSR 写入；新增 instruction/load/store page fault 接入和数据访存 fault 后的 WB trap 清流水。
- `vsrc/include/common.sv`: 扩展 CBus/IBus/DBus 响应，增加 `page_fault`；扩展 CBus 请求，增加 `is_instr` 标记。
- `vsrc/util/MMU.sv`: 新增 fault 状态和 PTE 权限检查，不再将无效 PTE 转成物理地址 0；支持 Sv39 canonical 地址检查和巨页 PPN 对齐检查。
- `vsrc/util/IBusToCBus.sv`、`vsrc/util/DBusToCBus.sv`: 传递 `is_instr` 与 `page_fault`。
- `vsrc/mycpu_top.sv`、`vsrc/util/CBusToAXI.sv`、`vivado/src/with_delay/soc_top.sv`: 对真实内存响应源固定 `page_fault=0`。
- `vsrc/include/csr.sv`: 修正 SSTATUS_MASK，开放 SIE/SPIE/SPP 等 S 态状态位；设置 MEDELEG_MASK/MIDELEG_MASK，支持常见异常和 S 级中断委托。
- `vsrc/util/CBusArbiter.sv`: idle 状态下直接透传当前请求，同周期完成的请求不再进入 busy。
- Makefile: 新增 `test-labplus-2`、`test-labplus-3`、`test-labplus-4`。
- `ready-to-run/lab+/:` 补充官方 Lab+ 测试二进制和汇编反汇编文件。

6 测试结果

6.1 Lab+ atomic extension

```
make test-labplus-3
```

```
The image is ./ready-to-run/lab+/3/atomicity.bin
```

```
Core 0: HIT GOOD TRAP at pc = 0x800000dc
```

```
total guest instructions = 55
```

```
instrCnt = 55, cycleCnt = 249
```

6.2 Lab+ privfull/PMP

```
Test ecall_u [OK]
Test instr_misalign [OK]
Test instr_access_fault [OK]
Test illegal_instr [OK]
Test breakpoint [X]
Test load_misalign [OK]
Test load_fault [OK]
Test store_misalign [OK]
Test store_fault [OK]
Test timer_intr [OK]
Test software_intr [OK]
Test pmp_nr [OK]
Test pmp_nw [OK]
Test pmp_nx [OK]
Test mem_detect [OK]
    4096B user memory detected. (Interrupts: 27899)
Test m_trap [OK]
Privileged test finished.
Exit with code = 0
```

6.3 回归测试

```
Lab1 extra:
Core 0: HIT GOOD TRAP at pc = 0x8002001c
instrCnt = 32775, cycleCnt = 120425
```

```
Lab4:
Core 0: HIT GOOD TRAP at pc = 0x8001fff8
instrCnt = 32766, cycleCnt = 139050
```

```
Lab5:
xv6 kernel is booting
Return from init! Test passed
```

```
Lab6:
Test ecall_u [OK]
Test instr_misalign [OK]
Test load_misalign [OK]
Test store_misalign [OK]
Test timer_intr [OK]
Test software_intr [OK]
Test m_trap [OK]
Privileged test finished.
Exit with code = 0
```

6.4 S-mode/SRET 回归

本次新增 S-mode、委托路径、MMU page fault 与 PTE 权限检查后重新运行：

```
make test-labplus-3
Core 0: HIT GOOD TRAP at pc = 0x800000dc
instrCnt = 55, cycleCnt = 249

Lab5 kernel:
Return from init! Test passed

TEST=sys ./build/emu --no-diff -i ./ready-to-run/lab6/lab6-test.bin
Privileged test finished.
Exit with code = 0

TEST=sys ./build/emu --no-diff -i ./ready-to-run/lab+/4/all-test-privfull.bin
Privileged test finished.
Exit with code = 0
```

7 后续可做项

本次已经完成 xv6 主线的前两步：S-mode/trap delegation，以及 MMU page fault/PTE 基础权限检查。后续如果继续推进 xv6，需要补充 PTE A/D 位硬件更新或对应 trap 修复路径、SUM/MXR 对 S 态访问用户页和执行页读数据的影响、更有针对性的 page fault 单元测试、virtio 或简化磁盘 MMIO、S 态 timer/external interrupt 与 CLINT/PLIC 的转换路径，以及更完整的 PMP 多 entry 支持。

性能方向还可以继续实现 cache、分支预测或多级流水线，用于进一步提升 test-labplus-2。这些改动会触及 trap、CSR、MMU、取指和访存多个共享路径，风险明显高于本次 PMP0/atomic/前端空泡优化，因此本次没有继续扩大范围。

8 总结

本次 Lab+ 新增完成了 atomic extension、PMP0/privfull 支持、EBREAK 断点异常、FENCE/FENCE.I 兼容，并加入两项轻量性能优化。继续推进 xv6 主 Track 时，已完成 S-mode、SRET、trap delegation，以及 MMU page fault/PTE 基础权限检查。AMO 实现利用现有单发访存结构，将 AMO 指令拆成不可被其他指令插入的读-改-写序列，并为 LR.W/SC.W 添加 reservation 状态。性能优化将 Lab1 extra 周期数从 185976 降到 120425，将 Lab4 周期数从 208529 降到 139050。最终 atomic、Lab+ privileged sys-test、Lab1 extra、Lab4、Lab5、Lab6 均完成回归。

9 AI 使用说明

我作为本项目的主导人，负责实验方案制定、代码审阅、测试验证和最终提交确认。AI (Codex) 用于辅助阅读 Lab+ 要求、分析官方 atomicity.S 和 all-test-privfull.S、实现 AMO/PMP/性

能优化、定位 difftest atomic event 与 trap 重定向时序问题，并整理实验报告。最终代码和报告由我本人审阅确认。